

习题四

一、选择题

- 下面关于串的叙述中，哪一个是不正确的？（ ）
A . 串是字符的有限序列 B . 空串是由空格构成的串
C . 模式匹配是串的一种重要运算 D . 串既可以采用顺序存储，也可以采用链式存储
- 若串 $S_1 = \text{'ABCDEFG'}$, $S_2 = \text{'9898'}$, $S_3 = \text{'###'}$, $S_4 = \text{'012345'}$, 执行
 $\text{concat}(\text{replace}(S_1, \text{substr}(S_1, \text{length}(S_2), \text{length}(S_3)), S_3), \text{substr}(S_4, \text{index}(S_2, \text{'8'}), \text{length}(S_2)))$ 其结果为（ ）
A . ABC###G0123 B . ABCD###2345 C . ABC###G2345 D . ABC###2345
E . ABC###G1234 F . ABCD###1234 G . ABC###01234
- 设有两个串 p 和 q, 其中 q 是 p 的子串, 求 q 在 p 中首次出现的位置的算法称为（ ）
A . 求子串 B . 联接 C . 匹配 D . 求串长
- 已知串 $S = \text{'aaab'}$, 其 Next 数组值为（ ）。
A . 0123 B . 1123 C . 1231 D . 1211
- 串 'ababaaababaa' 的 next 数组为（ ）。
A . 012345678999 B . 012121111212 C . 011234223456 D . 0123012322345
- 设 S 为一个长度为 n 的字符串, 其中的字符各不相同, 则 S 中的互异的非平凡子串 (非空且不同于 S 本身) 的个数为（ ）。
A . $2n-1$ B . n^2 C . $(n^2/2) + (n/2)$ D . $(n^2/2) + (n/2) - 1$ E . $(n^2/2) - (n/2) - 1$ F . 其他情况
- 串的长度是指（ ）
A . 串中所含不同字母的个数 B . 串中所含字符的个数
C . 串中所含不同字符的个数 D . 串中所含非空格字符的个数

二、判断题

- KMP 算法的特点是在模式匹配时指示主串的指针不会变小。()
- 设模式串的长度为 m, 目标串的长度为 n, 当 $n \approx m$ 且处理只匹配一次的模式时, 朴素的匹配 (即子串定位函数) 算法所花的时间代价可能会更为节省。()
- 串是一种数据对象和操作都特殊的线性表。()

三、填空题

- 空格串是指 (1), 其长度等于 (2)。
- 组成串的数据元素只能是_____。
- 一个字符串中_____称为该串的子串。
- *. INDEX ('DATASTRUCTURE', 'STR') = _____。
- 设正文串长度为 n, 模式串长度为 m, 则串匹配的 KMP 算法的时间复杂度为_____。
- *. 模式串 $P = \text{'abaabcac'}$ 的 next 函数值序列为_____。
- 串是一种特殊的线性表, 其特殊性表现在 (1); 串的两种最基本的存储方式是 (2)、(3); 两个串相等的充分必要条件是 (4)。

四、应用题

- 两个字符串 S_1 和 S_2 的长度分别为 m 和 n. 求这两个字符串最大共同子串算法的时间复杂度为 $T(m, n)$. 估算最优的 $T(m, n)$, 并简要说明理由。
- 设主串 $S = \text{'xyxyxyxyxyxy'}$, 模式串 $T = \text{'xyxy'}$. 请问: 如何用最少的比较次数找到 T 在 S 中出现的位置? 相应的比较次数是多少?
- KMP 算法(字符串匹配算法)较 Brute(朴素的字符串匹配)算法有哪些改进?
- *. 已知模式串 $t = \text{'abcaabbabab'}$ 写出用 KMP 法求得的每个字符对应的 next 和 nextval 函数值。

5* . 设字符串 S='aabaabaabaac', P='aabaac'

(1) 给出 S 和 P 的 next 值和 nextval 值;

(2) 若 S 作主串, P 作模式串, 试给出利用 BF 算法和 KMP 算法的匹配过程。

6 . 设 S1,S2 为串, 请给出使 S1//S2=S2//S1 成立的所有可能的条件 (//为连接符)。

五、算法设计

1 . 输入一个字符串, 内有数字和非数字字符, 如: ak123x456 17960?302gef4563, 将其中连续的数字作为一个整体, 依次存放到一数组 a 中, 例如 123 放入 a[0],456 放入 a [1], ... 。编程统计其共有多少个整数, 并输出这些数。

2 . 函数 void insert(char*s,char*t,int pos)将字符串 t 插入到字符串 s 中, 插入位置为 pos。请用 c 语言实现该函数。假设分配给字符串 s 的空间足够让字符串 t 插入。(说明: 不得使用任何库函数)

3 . S="S1S2...Sn"是一个长为 N 的字符串, 存放在一个数组中, 编程序将 S 改造之后输出:

(1) 将 S 的所有第偶数个字符按照其原来的下标从大到小的次序放在 S 的后半部分;

(2) 将 S 的所有第奇数个字符按照其原来的下标从小到大的次序放在 S 的前半部分;

例如:

S='ABCDEFGHJKLM'

则改造后的 S 为'ACEGIKLJHFD'B'。

参考答案

一、选择题

1、B	2、E	3、C	4、A	5、C	6、D	7、B
-----	-----	-----	-----	-----	-----	-----

二、判断题

1√	2√	3√
----	----	----

三、填空题

1 . (1) 由空格字符 (ASCII 值 32) 所组成的字符串 (2)空格个数	2 字符
3 . 任意个连续的字符组成的子序列	4 . 5
5 . $O(m+n)$	6 . 01122312
7 . (1)其数据元素都是字符(2)顺序存储(3)和链式存储(4)串的长度相等且两串中对应位置的字符也相等	

四、应用题

1 . 最优的 $T(m,n)$ 是 $O(n)$ 。串 S2 是串 S1 的子串, 且在 S1 中的位置是 1。开始求出最大公共子串的长度恰是串 S2 的长度, 一般情况下, $T(m,n) = O(m*n)$ 。

2 . 朴素的模式匹配 (Brute - Force) 时间复杂度是 $O(m * n)$, KMP 算法有一定改进, 时间复杂度达到 $O(m + n)$ 。本题也可采用从后面匹配的方法, 即从右向左扫描, 比较 6 次成功。另一种匹配方式是从左往右扫描, 但是先比较模式串的最后一个字符, 若不等, 则模式串后移; 若相等, 再比较模式串的第一个字符, 若第一个字符也相等, 则从模式串的第二个字符开始, 向右比较, 直至相等或失败。若失败, 模式串后移, 再重复以上过程。按这种方法, 本题比较 18 次成功。

3 . KMP 算法主要优点是主串指针不回溯。当主串很大不能一次读入内存且经常发生部分匹配时, KMP 算法的优点更为突出。

4 . 模式串的 next 函数定义如下:

next [j] =

根据此定义, 可求解模式串 t 的 next 和 nextval 值如下:

j	1	2	3	4	5	6	7	8	9	10	11	12
t 串	a	b	c	a	a	b	b	a	b	c	a	b
next[j]	0	1	1	1	2	2	3	1	2	3	4	5
nextval[j]	0	1	1	0	2	1	3	0	1	1	0	5

5 . 1) S 的 next 与 nextval 值分别为 012123456789 和 002002002009, p 的 next 与 nextval 值分别为 012123 和 002003。

(2) 利用 BF 算法的匹配过程:

第一趟匹配: aabaabaabaac

aabaac(i=6,j=6)

第二趟匹配: aabaabaabaac

aa(i=3,j=2)

第三趟匹配: aabaabaabaac

a(i=3,j=1)

第四趟匹配: aabaabaabaac

aabaac(i=9,j=6)

第五趟匹配: aabaabaabaac

aa(i=6,j=2)

第六趟匹配: aabaabaabaac

a(i=6,j=1)

第七趟匹配: aabaabaabaac

(成功) aabaac(i=13,j=7)

利用 KMP 算法的匹配过程:

第一趟匹配: aabaabaabaac

aabaac(i=6,j=6)

第二趟匹配: aabaabaabaac

(aa)baac

第三趟匹配: aabaabaabaac

(成功) (aa)baac

6. (1) s1 和 s2 均为空串; (2) 两串之一为空串; (3) 两串串值相等 (即两串长度相等且对应位置上的字符相同)。
(4) 两串中一个串长是另一个串长 (包括串长为 1 仅有一个字符的情况) 的数倍, 而且长串就好像是由数个短串经过连接操作得到的。

第五部分、算法设计

1. [问题分析] 在一个字符串内, 统计含多少整数的问题, 核心是如何将数从字符串中分离出来。从左到右扫描字符串, 初次碰到数字字符时, 作为一个整数的开始。然后进行拼数, 即将连续出现的数字字符拼成一个整数, 直到碰到非数字字符为止, 一个整数拼完, 存入数组, 再准备下一整数, 如此下去, 直至整个字符串扫描到结束。

int CountInt ()

// 从键盘输入字符串, 连续的数字字符算作一个整数, 统计其中整数的个数。

{**int** i=0, a[]; // 整数存储到数组 a, i 记整数个数

scanf ("%c", &ch); // 从左到右读入字符串

while (ch!='#') // '#'是字符串结束标记

if (isdigit (ch)) // 是数字字符

{num=0; // 数初始化

while (isdigit (ch) && ch!='#') // 拼数

{num=num*10+'ch'-'0';

scanf ("%c", &ch); }

a[i]=num; i++;

if (ch!='#') scanf ("%c", &ch); // 若拼数中输入了'#', 则不再输入

} // 结束 **while** (ch!= '#')

printf ("共有%d 个整数, 它们是: "i);

for (j=0; j<i; j++)

{printf ("%6d", a[j]);

if ((j+1) %10==0) printf ("\n"); } // 每 10 个数输出在一行上

} // 算法结束

[算法讨论] 假定字符串中的数均不超过 32767, 否则, 需用长整型数组及变量。

2. [题目分析] 本题是字符串的插入问题, 要求在字符串 s 的 pos 位置, 插入字符串 t。首先应查找字符串 s 的 pos 位置, 将第 pos 个字符到字符串 s 尾的子串向后移动字符串 t 的长度, 然后将字符串 t 复制到字符串 s 的第 pos 位置后。

对插入位置 pos 要验证其合法性, 小于 1 或大于串 s 的长度均为非法, 因题目假设给字符串 s 的空间足够大, 故对插入不必判溢出。

void insert(char *s,char *t,int pos)

//将字符串 t 插入字符串 s 的第 pos 个位置。

```

{int i=1,x=0; char *p=s,*q=t; //p, q 分别为字符串 s 和 t 的工作指针
  if(pos<1) {printf("pos 参数位置非法\n");exit(0);}
  while(*p!='\0'&&i<pos) {p++;i++;} //查 pos 位置
    //若 pos 小于串 s 长度, 则查到 pos 位置时, i=pos。
  if(*p == '\0') {printf("%d 位置大于字符串 s 的长度",pos);exit(0);}
  else //查找字符串的尾
    while(*p!= '\0') {p++; i++;} //查到尾时, i 为字符'\0'的下标, p 也指向'\0'。
  while(*q!= '\0') {q++;x++;} //查找字符串 t 的长度 x, 循环结束时 q 指向'\0'。
  for(j=i;j>=pos;j--){*(p+x)=*p; p--;} //串 s 的 pos 后的子串右移, 空出串 t 的位置。
  q--; //指针 q 回退到串 t 的最后一个字符
  for(j=1;j<=x;j++) *p--=*q--; //将 t 串插入到 s 的 pos 位置上
[算法讨论] 串 s 的结束标记('\0')也后移了, 而串 t 的结尾标记不应插入到 s 中。

```

3. [题目分析]对读入的字符串的第奇数个字符, 直接放在数组前面, 对第偶数个字符, 先入栈, 到读字符串结束, 再将栈中字符出栈, 送入数组中。限于篇幅, 这里编写算法, 未编程序。

```

void RearrangeString()
//对字符串改造, 将第偶数个字符放在串的后半部分, 第奇数个字符前半部分。
{char ch,s[],stk[]; //s 和 stk 是字符数组 (表示字符串) 和字符栈
  int i=1,j; //i 和 j 字符串和字符栈指针
  while((ch=getchar())!='#')// '#'是字符串结束标志
    s[i++]=ch; //读入字符串
  s[i]='\0'; //字符数组中字符串结束标志
  i=1;j=1;
  while(s[i]) //改造字符串
    {if(i%2==0) stk[i/2]=s[i]; else s[j++]=s[i];
      i++; }//while
  i--; i=i/2; //i 先从'\0'后退, 是第偶数字符的个数
  while(i>0) s[j++]=stk[i--] //将第偶数个字符逆序填入原字符数组
}

```